

Report

Big Data Tools: Decision Tree and Logistic Regression on the KDD Dataset

Team 6:

Tan

Cheng

Zhixuan Wang

Overview:

We implemented two Spark ML programs. One uses the Decision Tree algorithm and the other uses the Logistic Regression algorithm. We applied both to the KDD dataset, which describes network connections and labels each connection as either a normal connection or an anomaly connection (an attack). We ran each program 10 times on the school's Hadoop cluster. Each run used a different seed (1 to 10) to split the data into a training set and a test set. This report describes the two programs with pseudocode, explains how to install and run them, reports the results of the 10 runs, and compares and discusses the two models.

The dataset and the task:

The KDD dataset has 2 classes and 41 features. The two classes are normal connection and anomaly connection. The 41 features describe each network connection, for example its duration, its protocol type, the number of bytes sent, and many rate based statistics. We treat the task as a binary classification problem. We map the label "normal" to 0 and every other label to 1. So 0 means a normal connection and 1 means an attack. Both programs share the same loading and preprocessing steps, so the only real difference between them is the model.

(a) Pseudocode

We firstly wrote DT and LR separately. Then we decided to combine them into one. We keep the loading and preprocessing in one shared block, then give one block for each model, and one block for the 10 seed evaluation loop. This matches how the real program is organised, so the same code serves both models.

Load and Preprocess (both)

```
function loadData(inputPath):  
    read inputPath as CSV with header = false and inferSchema = true  
    rename the columns to the 41 feature names plus the label column  
    print the row count  
    drop rows that contain null values
```

```

print the row count again
for each feature column:
    if its type is string, record that categorical data exists
create a new column labelIndex:
    if label starts with "normal" then 0.0
    else 1.0
return the DataFrame

```

Decision Tree

```

assembler = VectorAssembler(inputCols = the 41 features, outputCol =
"features")
dt = DecisionTreeClassifier(featuresCol = "features", labelCol =
"labelIndex")
pipelineDT = Pipeline(stages = [assembler, dt])

```

Logistic Regression

```

assembler = VectorAssembler(inputCols = the 41 features, outputCol =
"features")
lr = LogisticRegression(featuresCol = "features", labelCol = "labelIndex",
maxIter = 10, regParam = 0.0)
pipelineLR = Pipeline(stages = [assembler, lr])

```

10 seed evaluation loop (both)

```

function runTenTimes(pipeline, data):
    results = empty list
    for seed in [1 ... 10]:
        startTime = now()
        train, test = data.randomSplit([0.7, 0.3], seed = seed)
        cache train and test
        model = pipeline.fit(train)
        trainPred = model.transform(train)
        testPred = model.transform(test)
        trainAccuracy = evaluator.evaluate(trainPred)
        testAccuracy = evaluator.evaluate(testPred)
        elapsed = now() minus startTime
        append (seed, trainAccuracy, testAccuracy, elapsed) to results
    return results

```

The evaluator is a MulticlassClassificationEvaluator with metric = accuracy. After the loop we compute the max, min, average, and standard deviation of the train accuracy, the test accuracy, and the running time. The standard deviation is the population standard deviation.

(b) Readme.txt (how to install and run)

Below are the steps on how to run the program on the school's Hadoop cluster. We connect to the cluster, set up the Spark environment, put the data on HDFS, and submit the program. Replace \$USER_NAME with our own ECS user name.

Step 1. Connect to the cluster with two SSH hops.

```
ssh $USER_NAME@barretts.ecs.vuw.ac.nz
ssh co246a-1
```

After the second hop the prompt changes to co246a-1%.

Step 2. Set up the Spark environment. Copy SetupSparkClasspath.sh into your working directory and source it.

```
source SetupSparkClasspath.sh
```

Nothing is printed, so we verify it worked.

```
echo $HADOOP_VERSION
echo $PATH
```

Step 3. Get a Kerberos ticket so that HDFS works. The ticket lasts for the day, so we do this again each session.

```
kinit $USER_NAME
```

Step 4. Put the input data on HDFS. The program reads from HDFS, so kdd.data must be there first.

```
hdfs dfs -put kdd.data /user/$USER_NAME/kdd.data
```

(HDFS commands can be used:)

hdfs dfs -ls /user/\$USER_NAME	list your files
hdfs dfs -put filename path	upload a file
hdfs dfs -get filename	download a file
hdfs dfs -rm filename	delete a file

Step 5. Submit the program with spark-submit. We run it on YARN in cluster mode. We pass the JAVA_HOME for the cluster containers, the program file, the input path, and the output path.

```
spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --conf spark.yarn.appMasterEnv.JAVA_HOME=/usr/lib/jvm/java-21-openjdk \
  --conf spark.executorEnv.JAVA_HOME=/usr/lib/jvm/java-21-openjdk \
  A3CodeAIO.py \
  hdfs:///user/$USER_NAME/kdd.data \
  hdfs:///user/$USER_NAME/results
```

The program file A3CodeAIO.py can be a local path. spark-submit uploads it to HDFS automatically.

Step 6. Check that the job succeeded. In the YARN log, look for the line that says final status: SUCCEEDED. The word FINISHED on its own does not mean success, so we always check the final status.

Step 7. Read the results from HDFS. In cluster mode the driver runs inside a YARN container, so anything the program prints does not come back to the terminal. The numbers are written to HDFS instead, so we read them from there. Wrap each path in single quotes because the shell may try to expand the star by itself.

```
hdfs dfs -cat '/user/$USER_NAME/results/DecisionTree/summary/part-*'
hdfs dfs -cat '/user/$USER_NAME/results/LogisticRegression/summary/part-*'
hdfs dfs -cat '/user/$USER_NAME/results/DecisionTree/runs/part-*'
hdfs dfs -cat '/user/$USER_NAME/results/LogisticRegression/runs/part-*'
```

(c) Results

We ran each program 10 times in cluster mode with seeds 1 to 10. The tables below give the max, min, average, and standard deviation of the training accuracy, the test accuracy, and the running time in seconds.

Decision Tree, 10 runs

Metric	max	min	average	std
Train accuracy	0.959838	0.943267	0.954407	0.006202
Test accuracy	0.959802	0.941964	0.953356	0.006174
Running time (s)	3.07	1.16	1.43	0.55

Logistic Regression, 10 runs

Metric	max	min	average	std
Train accuracy	0.884558	0.881584	0.882664	0.000863

Metric	max	min	average	std
Test accuracy	0.887336	0.880811	0.883130	0.001743
Running time (s)	1.99	1.13	1.26	0.24

We also ran it separately on our accounts (three of us all ran successfully!) on the same cluster. The accuracy numbers came back identical to six decimal places. This confirms that the results are reproducible, because the seeds, the code, and the data are fixed, so the train and test split is certain. Only the running time changed a little, because it depends on how busy the cluster is.

(d) Comparison and discussion

Accuracy:

The Decision Tree is obviously better. On the KDD dataset the Decision Tree reaches an average test accuracy of 95.34%, while the Logistic Regression reaches 88.31%. The gap is quite big. We think the reason is the shape of the data. The 41 features interact in nonlinear ways, and the Decision Tree can capture these interactions because it splits the data step by step. The Logistic Regression is a linear model, so its decision boundary is a single flat surface, and this limits how high its accuracy can go on this data.

Stability:

The Logistic Regression is steadier though. Across the 10 seeds, the Logistic Regression has a test accuracy standard deviation of only 0.0017. The Decision Tree has 0.0062, which is about four times larger. So the Logistic Regression is less sensitive to how the data is split. This is a normal trade off: the Decision Tree gives higher accuracy but it varies more from run to run; the Logistic Regression gives lower accuracy but it is steadier and easier to reproduce.

Running time:

The two models take a similar amount of time. The Decision Tree averages 1.43 seconds per run and the Logistic Regression averages 1.26 seconds per run. The slowest single run is always the first one, seed 1, at about 3 seconds for the Decision Tree and 2 seconds for the Logistic Regression. This is because the Java Virtual Machine and the Spark engine need to warm up at the first try. After the first run each model settles to about 1 second. So the high maximum is a warm up cost and not the real cost of the model.

In conclusion, as we can see, the training accuracy and the test accuracy are very close for both models. The Decision Tree goes 0.9544 on train and 0.9534 on test, a gap of only about 0.001. For the Logistic Regression the test accuracy is even slightly higher than the training accuracy. So under the 70/30 split both models generalise well and we see no overfitting. One thing to notice is that we left the Decision Tree depth at the Spark default value and the Logistic Regression at `maxIter = 10` with `regParam = 0.0`, which we did not

tune. Tuning the tree depth in particular might reduce the run to run variance of the Decision Tree. The Decision Tree wins on accuracy and the Logistic Regression wins on stability. Both run quickly and neither overfits. If the goal is the highest accuracy on this dataset we would choose the Decision Tree. If the goal is a steady and very reproducible result we would choose the Logistic Regression.